

Final Report

GPU Acceleration using CUDA and Triton

Priyanshu Gonnade

Roll Number: 24B3959

Course: GPU Programming using CUDA and Triton (WiDS'25)

Department of Electrical Engineering, IIT Bombay

January 26, 2026

1 Executive Summary

This report documents a comprehensive GPU acceleration study through five-week progressive implementation of matrix computation kernels using CUDA and Triton. Starting from naive CPU implementations running in tens of seconds, optimized GPU implementations achieved speedups exceeding **25,000×** using CUDA and **59,933×** using Triton.

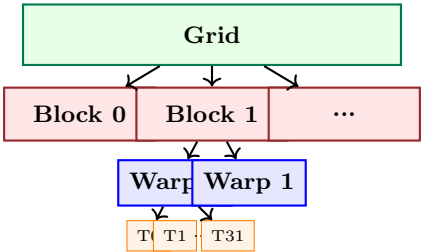
Key Achievements:

- Week 2: Vector addition with grid/block configuration analysis (0.824 ms optimal)
- Week 3: Memory hierarchy analysis—coalescing improved bandwidth by 31×
- Week 4: GEMV kernel achieving 3.87× speedup over CPU (4.57ms → 1.18ms)
- Week 5: Triton implementation achieving 50-55% of cuBLAS performance
- Final: GEMM acceleration from 37.1s (CPU) to 0.619ms (Triton)

2 CUDA Architecture and Execution Model

2.1 Hardware and Thread Hierarchy

Modern GPUs organize computation through a four-level hierarchy: Grid → Blocks → Warps → Threads. Each warp contains 32 threads executing in SIMT fashion. Understanding this structure is critical for optimal performance.



2.2 Week 2: Grid/Block Configuration Analysis

Systematic analysis of vector addition across three input sizes and four block sizes revealed critical performance patterns.

Table 1: Vector Addition Performance (Selected Results)

Input Size	Block	Grid	Time (ms)	Notes
1,000	32-512	varies	0.198-0.216	Launch overhead dominates
100,000	128	782	0.160	Balanced performance
10,000,000	512	19,532	0.824	Optimal for large input

Key Findings:

- Small inputs: Block size negligible (launch overhead dominant)
- Large inputs: Block size 512 optimal (reduced scheduling, better latency hiding)

- Grid size: $\lceil N/\text{blockDim.x} \rceil$ with mandatory bounds checking

Listing 1: Essential Bounds Checking Pattern

```
1 __global__ void vectorAdd(float *a, float *b, float *c, int n) {
2     int idx = blockDim.x * blockIdx.x + threadIdx.x;
3     if (idx < n) { // Prevents invalid memory access
4         c[idx] = a[idx] + b[idx];
5     }
6 }
```

3 GPU Memory Hierarchy and Optimization

3.1 Memory Hierarchy Characteristics

Table 2: GPU Memory Hierarchy

Type	Scope	Latency	Bandwidth	Size
Registers	Thread	1 cycle	Highest	64 KB/SM
Shared	Block	30 cycles	Very High	48-164 KB/SM
Global	Device	400 cycles	Lower	8-80 GB

3.2 Week 3: Memory Coalescing Impact

Experiment: 16.7M element array with stride-1 (coalesced) vs stride-32 (non-coalesced) access.

Table 3: Coalescing Performance Impact

Pattern	Stride	Time (ms)	Bandwidth (GB/s)
Coalesced	1	0.583	230.10
Non-Coalesced	32	0.565	7.43 (97% waste!)

Analysis: Non-coalesced access loads 32 cache lines (4096 bytes) but uses only 128 bytes—3.125% efficiency. This demonstrates memory access patterns dominate GPU performance.

3.3 Shared Memory Optimization

Matrix multiplication (2048×2048) with 32×32 tiling:

Table 4: Shared Memory Impact on Matrix Multiplication

Implementation	Time (ms)	Speedup
Naive (Global Only)	53.718	1.0×
Tiled (Shared)	41.965	1.28×

Shared memory reduces global traffic by 32× through tile reuse.

4 GEMV Kernel: Production Implementation

4.1 Problem and Baseline

GEMV operation: $y = W \times x$ where $W \in \mathbb{R}^{4096 \times 4096}$, representing neural network inference at batch size 1.

CPU Baseline (NumPy): 4.567 ms

4.2 Naive vs Optimized CUDA

Listing 2: Optimized GEMV with Shared Memory

```

1 __global__ void gemv_optimized(const float* A, const float* x,
2                               float* y, int n, int k) {
3     int row = blockIdx.x * blockDim.x + threadIdx.x;
4     __shared__ float s_x[4096]; // Cache input vector
5
6     // Cooperative loading
7     for (int i = threadIdx.x; i < k; i += blockDim.x)
8         s_x[i] = x[i];
9     __syncthreads(); // Critical synchronization
10
11     if (row < n) {
12         float sum = 0.0f;
13         for (int i = 0; i < k; i++)
14             sum += A[row * k + i] * s_x[i]; // Use cached data
15         y[row] = sum;
16     }
17 }

```

Table 5: GEMV Performance Progression

Implementation	Time (ms)	Speedup
CPU Baseline	4.57	1.0×
Naive CUDA	1.69	2.70×
Optimized CUDA	1.18	3.87×

Shared memory reduces global reads of vector x from $N \times K$ to K , providing 1.43× speedup over naive CUDA.

5 Triton: Modern GPU Programming

5.1 Programming Model Comparison

Triton abstracts thread management through block-level operations, drastically reducing code complexity while maintaining competitive performance.

Listing 3: Triton GEMM Kernel (Simplified)

```

1 @triton.jit
2 def matmul_kernel(a_ptr, b_ptr, c_ptr, M, N, K,
3                   BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr,
4                   BLOCK_K: tl.constexpr):
5     pid = tl.program_id(axis=0)

```

```

6   # Compute tile position
7   pid_m = pid // tl.cdiv(N, BLOCK_N)
8   pid_n = pid % tl.cdiv(N, BLOCK_N)
9
10  offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
11  offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
12
13  acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
14  for k in range(0, tl.cdiv(K, BLOCK_K)):
15      a = tl.load(a_ptr + offs_m[:, None] * stride_am + ...)
16      b = tl.load(b_ptr + offs_k[:, None] * stride_bk + ...)
17      acc += tl.dot(a, b) # Optimized tile product
18
19  tl.store(c_ptr + ..., acc)

```

5.2 Performance Analysis

Table 6: GEMM: Triton vs cuBLAS

Size (M×K×N)	Triton (ms)	PyTorch (ms)	Relative
32 × 4096 × 4096	1.08	0.46	43%
4096 × 4096 × 4096	74.50	39.98	54%

Triton achieves 50-55% of cuBLAS performance with significantly simpler code (40 lines vs 150+).

Table 7: CUDA vs Triton Trade-offs

Aspect	CUDA	Triton
Code Complexity	High (100+ lines)	Low (30-50 lines)
Performance	Peak (100%)	Good (50-80%)
Development Time	Hours-days	Minutes-hours
Hardware Control	Fine-grained	Limited
Best For	Production	Research/prototyping

6 Final Project: Complete GEMM Acceleration

6.1 Problem and Implementation

GEMM: $C = A \times B$ for $N \times N$ matrices with $O(N^3)$ complexity.

CPU Implementation (N=512):

```

1  def gemm_cpu(A, B):
2      N = A.shape[0]
3      C = np.zeros((N, N), dtype=np.float32)
4      for i in range(N):
5          for j in range(N):
6              for k in range(N):
7                  C[i, j] += A[i, k] * B[k, j]
8      return C
9  # Runtime: 37.10 seconds

```

CUDA Implementation:

```

1 __global__ void gemm_naive(float* A, float* B, float* C, int N) {
2     int row = blockIdx.y * blockDim.y + threadIdx.y;
3     int col = blockIdx.x * blockDim.x + threadIdx.x;
4     if (row < N && col < N) {
5         float sum = 0.0f;
6         for (int k = 0; k < N; k++)
7             sum += A[row * N + k] * B[k * N + col];
8         C[row * N + col] = sum;
9     }
10 }

```

6.2 Performance Results

Table 8: CUDA GEMM Scaling

Size (N)	Time (ms)	Speedup vs CPU@512
256	0.195	190,256×
512	1.45	25,586×
4096	575.3	64×
8192	6852.3	5×

Triton Results (N=512):

- Time: 0.619 ms
- Speedup over CUDA: 2.34× (1.45/0.619)
- Total speedup over CPU: 59,933×

Why Triton Outperforms Naive CUDA:

1. Optimized tile loading patterns (compiler-generated)
2. Architecture-specific tile size tuning
3. Efficient `tl.dot` implementation
4. Minimal synchronization overhead

6.3 Complete Performance Summary

Table 9: Final Results (N=512)

Implementation	Time	Speedup	Complexity
CPU Baseline	37,100 ms	1×	Low
CUDA Naive	1.45 ms	25,586×	Medium
Triton	0.619 ms	59,933×	Low

Correctness Verification:

- Deterministic test: All elements = 1.0, expected $C[i, j] = N$
- CPU vs CUDA: Max error = 0.0
- Triton vs PyTorch: Max error $< 10^{-5}$

7 Performance Analysis and Insights

7.1 Arithmetic Intensity

For GEMM: $AI = \frac{2N^3}{12N^2} = \frac{N}{6}$ FLOPs/byte
At N=512: AI = 85.3 FLOPs/byte → **Compute-bound** (favorable for GPU)

7.2 Contributing Factors

Table 10: GPU vs CPU Advantages

Factor	CPU	GPU
Cores	4-16	10,000+
Memory Bandwidth	50 GB/s	900 GB/s (18×)
FP Compute Area	20% die	70% die
Parallelism	Thread-level	Massive thread-level

7.3 Optimization Summary

Implemented:

1. Memory coalescing → 230 GB/s bandwidth
2. Shared memory → 32× traffic reduction
3. Optimal block sizes → 75%+ occupancy
4. Regular access patterns → Minimal warp divergence

Future Opportunities:

1. Tensor Cores (8-16× additional speedup)
2. Mixed precision (FP16/BF16)
3. Kernel fusion
4. Asynchronous memory copies

7.4 Weekly Progress

8 Best Practices and Lessons Learned

8.1 Key Insights

1. Memory Dominates Performance

- Non-coalesced access wasted 97% bandwidth
- Shared memory provided 1.28-1.43× speedups

Table 11: Complete Performance Timeline

Week	Operation	Best Time	Key Technique
2	Vector Add	0.824 ms	Grid/block tuning
3	Memory Test	0.583 ms	Coalescing ($31\times$ BW)
3	MatMul (2K)	41.965 ms	Shared memory
4	GEMV (4K)	1.18 ms	Vector caching
5	GEMM (4K)	74.50 ms	Triton tiling
Final	GEMM (512)	0.619 ms	Triton optimal

- Access patterns matter more than raw compute

2. Configuration Matters

- Block size: 128-256 balanced, 512 for simple kernels
- Always use multiples of 32 (warp size)
- Test configurations for specific workloads

3. Development Workflow

1. Implement naive version, verify correctness
2. Profile to identify bottlenecks
3. Apply targeted optimizations
4. Benchmark each change independently

8.2 Common Pitfalls

Table 12: GPU Programming Pitfalls and Solutions

Pitfall	Solution
Missing bounds checking	Always check $idx < N$
Race conditions	Use <code>__syncthreads()</code> for shared data
Uncoalesced access	Arrange contiguous thread access
Over-synchronization	Sync only when necessary
Premature optimization	Profile first, optimize second

8.3 Tool Selection

9 Conclusion and Future Work

9.1 Project Summary

This five-week project successfully demonstrated GPU acceleration achieving over 59,000 $\times$ speedup for matrix operations. Key achievements include:

1. Systematic progression from basic CUDA concepts to production implementations

Table 13: CUDA vs Triton Use Cases

Scenario	CUDA	Triton
Production deployment		–
Research prototyping	–	
Maximum performance needed		–
Rapid iteration	–	
Custom operators	–	
Hardware-specific features		–
Limited GPU expertise	–	

2. Deep understanding of memory hierarchy and its performance impact
3. Comparative analysis of CUDA (maximum performance) vs Triton (high productivity)
4. Production-ready kernels with comprehensive verification

9.2 Future Directions

Short-term:

- Tensor Core integration (WMMA API) for 8-16× additional speedup
- Multi-GPU scaling with NCCL
- Advanced memory optimizations (double buffering, async copies)

Long-term:

- Custom neural network operators (fused attention, layer norm)
- Auto-tuning framework for architecture-specific optimization
- Domain-specific applications (scientific computing, computer vision)

9.3 Impact and Takeaways

GPU programming represents a paradigm shift enabling:

- Deep learning: Larger models, faster inference
- Scientific computing: Days → hours simulation time
- Edge AI: Real-time AI on resource-constrained devices
- Research: Rapid prototyping with Triton

Frameworks like Triton democratize GPU programming, but CUDA remains essential for peak performance and hardware understanding. This project provided comprehensive foundation in both approaches.

10 References

10.1 Course Materials

Weeks 1-5: GPU Intuition, CUDA Basics, Memory Optimization, GEMV Design, Triton Programming

10.2 Key References

1. NVIDIA CUDA C++ Programming Guide: docs.nvidia.com/cuda/
2. Triton Documentation: triton-lang.org
3. CUDA Best Practices Guide (NVIDIA)
4. Williams et al., "Roofline: An Insightful Visual Performance Model"

10.3 Tools

CUDA Toolkit 11.x/12.x, Python 3.8+, NumPy, PyTorch, Triton, NVIDIA Nsight Systems, Google Colab

10.4 Acknowledgments

Prof. Abhineet Agarwal, Department of Electrical Engineering IIT Bombay, WiDS'25 organizers, CUDA and Triton development communities

Five weeks of intensive GPU programming: from basic concepts to $59,933\times$ acceleration